

Lab 12: Graphs and Shortest Paths

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Graph ADT, BFS, DFS, Dijkstra's algorithm

Objectives

- Represent a weighted directed graph using an adjacency list.
- Implement Breadth-First Search (BFS) for unweighted shortest paths.
- Implement Depth-First Search (DFS) for reachability and cycle detection.
- Implement Dijkstra's algorithm for single-source shortest paths on weighted graphs.

Quick Reference

Graph ADT.

```
// Vertex and Edge as objects
interface Vertex<V> { V element(); }
interface Edge<E>    { E element(); Vertex[] endVertices(); }
// Graph methods: vertices(), edges(), insertVertex(v), insertEdge(u,v,w),
//                outgoingEdges(u), incomingEdges(u), endVertices(e), opposite(u,e)
//                )
```

Adjacency list. Map each **Vertex** to a list of **Edge** objects. Space: $O(V + E)$.

BFS. Use a queue; mark vertices *discovered* on first visit. Finds shortest (hop-count) path in an unweighted graph.

Dijkstra's algorithm. Use a priority queue (min-heap on tentative distance $d[v]$). On relaxing edge (u, v, w) : if $d[u] + w < d[v]$, update $d[v]$ and decrease key. **Assumption:** non-negative edge weights. **Complexity:** $O((V + E) \log V)$ with a binary heap.

Quick Experiments (Try It)

Try It 1: Directed vs. undirected graphs

Build a four-vertex directed graph with edges $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$, $A \rightarrow D$. Print `outgoingEdges(A)` and `incomingEdges(A)`. Now build the same graph as undirected (each edge goes both ways). How does the adjacency list differ?

Try It 2: BFS tree depth

Build the undirected graph: vertices 0–5, edges (0,1), (0,2), (1,3), (1,4), (2,5). Run BFS from vertex 0. Record the layer (depth) of each vertex. What is the shortest path (in hops) from 0 to 5?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Build a Graph with Adjacency List (30 min)

Implement `AdjacencyListGraph<V,E>` as a directed graph:

- Vertices stored in a `Map<String, Vertex<V>>` (keyed by label).
- Each vertex has an outgoing edge list: `Map<Vertex, Edge<E>>`.
- `insertVertex(String label, V element)`, `insertEdge(String from, String to, E weight)`.

- `outgoingEdges(Vertex v), int numVertices(), int numEdges()`.
 - `printAdjList()`: print each vertex and its outgoing neighbours with weights.
- Build the 6-vertex weighted graph from lecture and print its adjacency list.

In-Lab Exercise 2: Breadth-First Search (25 min)

Implement `Map<Vertex, Integer> bfs(Vertex start)` that returns a map from each reachable vertex to its shortest hop-distance from `start`. Use a `Queue<Vertex>` and a `Set<Vertex>` `visited`.

- Also record the *BFS tree* (predecessor map) so you can reconstruct the path.
- Add `List<Vertex> shortestPath(Vertex start, Vertex end)` that returns the hop-count shortest path using the BFS predecessor map.

Test on your graph from Exercise 1.

In-Lab Exercise 3: Depth-First Search (20 min)

Implement recursive `void dfs(Vertex v, Set<Vertex> visited, List<Vertex> order)` that performs a DFS and appends each fully-processed vertex to `order` (post-order = topological sort for a DAG).

- Run DFS from every unvisited vertex to handle disconnected graphs.
- Detect a back edge (edge to a currently-active vertex) to detect cycles. Print “Cycle detected” if one is found.

In-Lab Exercise 4: Dijkstra’s Algorithm (35 min)

Implement `Map<Vertex,Integer> dijkstra(Vertex source)` using your `HeapPriorityQueue` from Lab 9 (or `java.util.PriorityQueue`).

1. Initialise $d[\text{source}] = 0$, all others $= \infty$ (`Integer.MAX_VALUE`).
2. Insert all vertices into the PQ with their initial distances as keys.
3. Each iteration: extract the vertex u with minimum $d[u]$; for each outgoing edge (u, v, w) : if $d[u] + w < d[v]$, update $d[v]$ and re-insert (or decrease key).

Run on the weighted directed graph from lecture. Print $d[v]$ for all vertices and the shortest-path tree (predecessor map). Verify against the hand-traced example.

AI Collaboration

Suggested prompts:

- “Why does Dijkstra’s algorithm fail with negative edge weights? Give a minimal counter-example.”
- “Explain the difference between BFS and DFS tree edges, back edges, and cross edges.”

Verify: Ask the AI to trace Dijkstra’s algorithm on a 5-vertex graph of your choice. Run your implementation on the same graph and compare the $d[v]$ values and predecessor map.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: BFS layering

Run BFS from vertex A on the undirected graph with edges: A-B, A-C, B-D, B-E, C-F, D-G. List the BFS discovery order and the layer (distance from A) of each vertex. Draw the BFS tree.

Trace & Predict 2: Dijkstra trace

Trace Dijkstra's algorithm from source s on the graph below. For each extraction from the PQ, record: vertex extracted, its final d value, and which neighbours are relaxed.

Edge	Weight
$s \rightarrow a$	4
$s \rightarrow b$	2
$b \rightarrow a$	1
$b \rightarrow c$	5
$a \rightarrow c$	1

What is the shortest path and distance from s to c ?

Write Code on Paper 1: Has path? using DFS

Write a method `boolean hasPath(String from, String to)` that returns `true` if there is a directed path from vertex `from` to vertex `to` in the graph. Use an iterative DFS with an explicit `Stack<Vertex>`.

Draw on Paper 1: Adjacency list vs. adjacency matrix

For the directed graph with vertices $\{1, 2, 3, 4\}$ and edges $1 \rightarrow 2$, $1 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 2$, $4 \rightarrow 3$: (a) Draw the adjacency list representation. (b) Draw the 4×4 adjacency matrix. (c) Label the space complexity of each. (d) State which is faster for `hasEdge(u,v)` and which is faster for listing all outgoing edges of u .

Submission. Save files as `lab13_ex*.java` and submit on the course site.